



KING SALMAN
INTERNATIONAL UNIVERSITY
FACULTY OF COMPUTER SCIENCE
AND ENGINEERING

CSC491 - GRADUATION PROJECT

Project Title

**EGX360: Intelligent Stock Market
Analysis and Forecasting Platform**

A Thesis Submitted To Computer Science Department
Faculty of Computer Science and Engineering, KSIU

In Partial Fulfillment of **B.Sc.** Degree in Computer Science Program

By

Mohamed Taher Amin

Supervisors

Dr. Samy Abdelnabi

Associate Professor, Faculty of Computer Science
King Salman International University

EGYPT, 2026



ACKNOWLEDGEMENT

I would like to express my sincere thanks to my supervisor, **Dr. Samy Abdelnabi**, for his continuous guidance and help throughout this project. I also want to thank **Eng. Abdelaziz Hussein** for his support, effort, and valuable advice which helped me a lot in completing this work.

Special thanks to the **Faculty of Computer Science and Engineering** at **King Salman International University** for providing the required resources and a great learning environment.

ABSTRACT

This thesis presents **EGX 360**, an AI-driven FinTech platform designed to mitigate volatility and data fragmentation in the Egyptian Exchange (EGX). The study demonstrates that integrating macroeconomic features (USD/EGP, gold, interest rates) with a Stacking Ensemble architecture (XGBoost & LightGBM) significantly enhances prediction accuracy. By utilizing Target Engineering (EMA10 direction), the model achieved an **85.22% accuracy and 80% down-trend recall on 28 years of data, yielding a +159% alpha** over the market baseline.

The ecosystem utilizes a scalable AWS and Supabase backend with autonomous scrapers and real-time WebSocket streams. It features an advanced bilingual NLP sentiment pipeline and a cross-platform Flutter frontend built on Feature-First Clean Architecture. Key functionalities include interactive charting, a Cerebras API Llama 3.1-powered AI assistant, a paper-trading simulator with a dynamic Guardian Engine for risk management, and a social trading community. Ultimately, EGX 360 provides a production-ready environment for independent investors to achieve data-driven financial autonomy.



LIST OF FIGURES

FIGURE 1.1 THE FUTURE VISION FOR AGRICULTURE [6]..... 1

FIGURE 1.2 TYPES OF AGRICULTURE ROBOTS..... 1

FIGURE 2.1 SOLAR-POWERED AGRICULTURE ROBOT DESIGN [2]..... 4





LIST OF TABLES

TABLE 1.1 PROPERTIES OF STAINLESS STEEL.....2





LIST OF SYMBOLS





TABLE OF CONTENTS

ACKNOWLEDGEMENT.....I

ABSTRACT..... II

LIST OF FIGURES..... III

LIST OF TABLES..... V

LIST OF SYMBOLS..... VII

TABLE OF CONTENTS..... IX

1. CHAPTER 1: INTRODUCTION..... 1

1.1. Introduction..... 1

1.2. Problem Statement..... 2

1.3. Objectives..... 2

1.4. Project Scope..... 3

2. LITERATURE REVIEW.....4

2.1. Financial Prediction Methods Overview.....5

2.2. Sentiment Analysis and NLP in Finance.....6

2.3. Emerging Markets vs Developed Markets..... 7

2.4. Risk Management in Automated Trading.....8

3. SYSTEM DESIGN & ARCHITECTURE 9

3.1. Requirements (Functional & Non-Functional) & Use Cases Diagram.....10

3.2. High-Level Architecture Diagram.....13

3.3. Database Schema and Conceptual Model.....14

3.4. Sequence Diagrams..... 15

3.5. Comprehensive Service-Level Architecture.....18

3.6. Activity Diagram.....19

3.7. Technology Stack Selection.....21

4. BACKEND INFRASTRUCTURE & DATA ENGINEERING.....22



4.1.	Data Sourcing & Scraper Engineering.....	23
4.1.1.	EGX Data via Reverse Engineering.....	23
4.1.2.	Real-Time Crypto Streaming (Non-Persistent).....	25
4.1.3.	Local Commodities & Market Summary Scraper.....	26
4.1.4.	Autonomous News Scraping & Anti-Bot Strategies.....	26
4.2.	Data Processing & NLP Pipeline.....	28
4.2.1.	Text Cleaning & Metadata Extraction.....	28
4.2.2.	Arabic Sentiment Analysis (Cerebras & Llama 3.1).....	28
4.2.3.	English Sentiment Analysis (FinBERT)	28
4.3.	Database Architecture & Security.....	30
4.3.1.	Unified Entity-Relationship Diagram (ERD) Strategy.....	30
4.3.2.	Database Optimization (Normalization, Indexes, & Triggers).....	33
4.3.3.	Encrypted Virtual Wallet & Transaction Ledger.....	34
4.3.4.	Security & Row Level Security (RLS) & Policies	35
4.3.5.	Supabase RPCs (Advanced Analytics).....	36
4.4.	Authentication & User Access Control.....	36
4.4.1.	OAuth 2.0 & Social Integration.....	36
4.4.2.	SMTP Relay & Deep-Linking.....	37
4.4.3.	Secure User State Management.....	37
4.5.	Cloud Orchestration & Automation.....	38
4.5.1.	AWS EC2 Deployment & Environment Setup.....	38
4.5.2.	Automated Daemons & Market Status Tracker.....	38
4.5.3.	Guardian Engine (Risk Management Logic)	39
5.	INTELLIGENT PLATFORM SERVICES.....	13
5.1.	Intelligent AI Chatbot & Financial RAG Engine.....	
5.1.1.	LLM Orchestration (Cerebras API).....	12
5.1.2.	Semantic Retrieval & Portfolio Context Awareness.....	12
5.1.3.	Real-Time Data Injection Pipeline	12

5.1.4.	Chat Session Memory & Persistence.....	12
5.2.	Social & Real-Time Interaction Infrastructure.....	
5.2.1.	Community Feed & Live Chat.....	12
5.2.2.	Supabase Storage & Object Management.....	12
5.2.3.	Content-User Linking.....	12
5.2.4.	User Sentiment & Crowdsourced Analysis.....	12
5.3.	Unified Notification & Analytical Event System.....	
5.3.1.	Firebase Cloud Messaging (FCM) Integration.....	12
5.3.2.	Scraper & Analytics Triggers.....	12
5.3.3.	Portfolio Watchlist-Driven Alerts	12
6.	QUANTITATIVE AI MODEL DEVELOPMENT.....	13
6.1.	Market Randomness & Target Engineering.....	13
6.1.1.	The Random Walk Challenge in EGX30	12
6.1.2.	Market Noise Extraction & SNR Analysis.....	12
6.1.3.	EMA10 Direction Target Engineering.....	12
6.2.	Advanced Feature Engineering & Macro Integration.....	13
6.2.1.	Technical Indicators & Momentum Metrics.....	12
6.2.2.	Egypt-Specific Macro Features (USD/EGP, Gold, Interest Rates).....	12
6.2.3.	Temporal Features & Cyclic Time Encoding	12
6.3.	Model Architecture & Ensemble Strategy.....	13
6.3.1.	Baseline Model: LSTM Network Architecture.....	12
6.3.2.	Production Model: Stacking Ensemble Framework.....	12
6.3.3.	Base Learners (XGBoost & LightGBM) & Meta-Learner Integration....	12
6.3.4.	Hyperparameter Optimization via Optuna Bayesian Search	12
6.4.	Model Evaluation & Financial Backtesting.....	13
6.4.1.	Training Strategy & Time Series Cross-Validation.....	12
6.4.2.	Low Liquidity Adjustments.....	
6.4.3.	Predictive Accuracy & Classification Metrics.....	12
6.4.4.	Class Imbalance Handling & Down-Trend Detection.....	12

6.4.5. Signal Lagging & Look-Ahead Bias Prevention.....12

6.4.6. Trading Simulation & Alpha Generation.....12

6.5. Cross-Market Applicability.....13

6.5.1. TrCryptocurrency Market Adaptation (BTC/USD).....12

6.5.2. Crypto Model Evaluation & Results.....

7. FRONTEND IMPLEMENTATION & USER EXPERIENCE.....16

7.1. Flutter Multi-Platform Strategy (Web, Android, Desktop).....16

7.1.1. Flutter Multi-Platform Strategy (Web, Android, Desktop)12

7.1.2. Responsive Design System & RTL Localization12

7.1.3. Design Tokens & UI Performance.....12

7.2. State Management & Architecture Patterns16

7.2.1. Feature-First Clean Architecture12

7.2.2. GetX Reactive State Management12

7.2.3. Local Data Persistence (Floor SQLite & GetStorage)12

7.2.4. Real-Time Data Routing (WebSocket & Polling)12

7.4. Core Application Features.....16

7.4.1. Advanced Interactive Charting System12

7.4.2. AI-Powered Financial Chatbot & Gauge12

7.4.3. Social Trading & Real-Time Community12

7.4.4. Paper Trading Simulator12

7.4.5. Risk Management & Guardian Dashboard12

7.5. Navigation & User Experience.....16

7.5.1. 29 Named Routes System12

7.5.2. Desktop & Mobile Navigation Patterns12

7.5.3. Deep-Linking & OAuth Integration12

8. SYSTEM TESTING & OVERALL EVALUATION.....19

9. CONCLUSION & FUTURE WORK.....19

References.....21

Appendix..... 21



CHAPTER 1

INTRODUCTION

1.INTRODUCTION

1.1. Introduction

The Egyptian Exchange (EGX) is a cornerstone of the national economy, acting as a primary barometer for investment and corporate growth in the region. As the financial landscape undergoes a rapid digital transformation, the role of Artificial Intelligence (AI) has become indispensable in interpreting market complexities. **EGX 360** is an intelligent, multi-platform financial ecosystem designed to empower individual investors by bridging the gap between raw market data and actionable insights. By integrating advanced quantitative modeling with modern software engineering, the project provides a unified environment for market prediction, automated analysis, and risk-managed trading simulation.

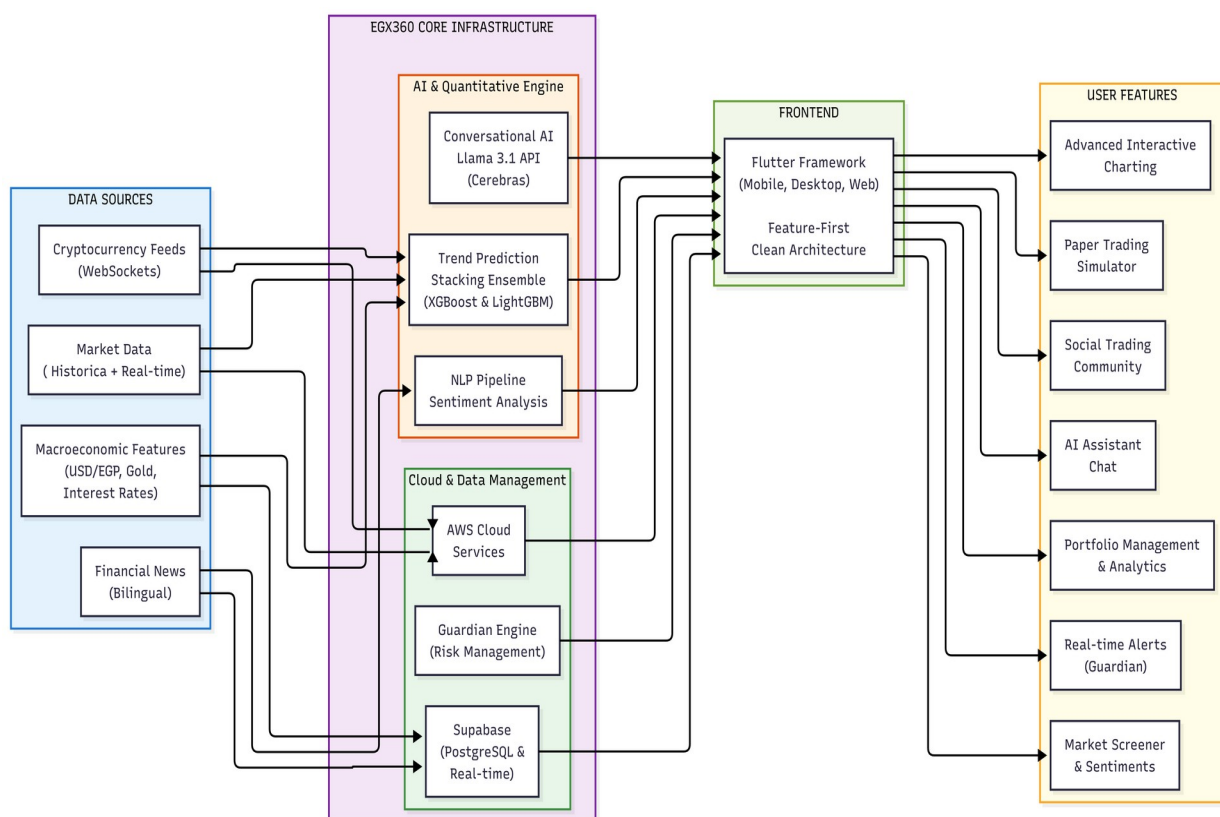


FIGURE 1.1: HIGH-LEVEL CONCEPT OF THE EGX 360 ECOSYSTEM

1.2. Problem Statement

Despite the availability of market data, independent retail investors in Egypt face significant barriers that hinder effective decision-making:

1. **Data Fragmentation:** Critical market variables, including technical indicators, localized news, and macroeconomic factors (USD/EGP, gold, interest rates), are scattered across disparate sources, leading to information overload and analytical paralysis.
2. **Extreme Non-Stationarity (EGX30 Noise):** Emerging markets exhibit high volatility where asset prices are non-linearly driven by macroeconomic shifts. Traditional forecasting models fail because they rely solely on historical price action without integrating macro-variables and real-time sentiment.
3. **Bilingual Financial NLP Bottlenecks:** Local financial news is highly bilingual (Arabic and English). Standard English models lack localized market context, while Arabic financial text suffers from a severe lack of high-quality, pre-trained sentiment analysis models.
4. **Lack of Production-Grade Platforms:** There is a notable absence of localized platforms that integrate real-time data ingestion, conversational AI, and server-side risk management (e.g., automated stop-loss triggers) in a single secure environment.

1.3. Objectives

The primary goal of this project is to build a robust, production-grade financial intelligence platform that bridges the gap between complex data and individual investors. The specific objectives include:

- **Multimodal Quantitative Forecasting:** Developing a sophisticated predictive engine using a Stacking Ensemble architecture (XGBoost and LightGBM). This objective extends beyond price history to include the integration of macroeconomic features (USD/EGP rates, gold prices, interest rates) and bilingual NLP sentiment analysis (Arabic and English) to achieve high classification accuracy and market alpha.

- **Production-Grade Software Architecture:** Engineering a high-performance, cross-platform Flutter application using a "Feature-First Clean Architecture" to ensure system scalability, high testability, and seamless multi-platform deployment.
- **Autonomous Data Orchestration:** Establishing a resilient backend infrastructure on AWS and Supabase. This includes building autonomous data scrapers for the Egyptian Exchange (EGX) and real-time WebSocket streams for cryptocurrency markets to ensure zero-latency data availability.
- **Intelligent Investor Ecosystem:** Integrating advanced AI services, including a real-time financial chatbot integrating the Llama 3.1 model via the Cerebras API for contextual market support. Additionally, the project aims to deploy a "Guardian Engine" for automated risk management and a real-time social community to foster interactive trading.
- **Risk-Free Financial Simulation:** Creating a high-fidelity "Paper Trading" environment that allows users to test strategies using real market data without financial risk, supported by automated portfolio analytics and performance tracking.

1.4. Project Scope

The scope of EGX 360 is defined by the following technical and functional boundaries:

- **Target Markets:** Focuses primarily on the Egyptian Stock Exchange (EGX30 and EGX70 indices) and major Cryptocurrency pairs (e.g., BTC/USD).
- **Data Horizon:** Utilizes over 28 years of historical EGX data, supplemented by real-time macroeconomic features such as USD/EGP exchange rates, gold prices, and national interest rates.
- **Technological Infrastructure:** Deployment on AWS cloud infrastructure, utilizing Supabase for secure backend management and real-time data streaming.
- **Methodology:** Implementation of "Feature-First Clean Architecture" to ensure system scalability and maintainability.

CHAPTER 2

LITERATURE REVIEW

2.LITERATURE REVIEW

This chapter reviews the evolution of financial prediction methodologies, from traditional statistical models to advanced Machine Learning (ML) and Deep Learning (DL) architectures. It analyzes the limitations of raw price prediction in emerging markets and highlights the gap in Arabic sentiment analysis. Finally, it establishes the rationale for the predictive architecture and risk management strategies employed in the EGX360 ecosystem.

2.1. Financial Prediction Methods Overview

Financial time series prediction has evolved from traditional statistical models (ARIMA) to DL architectures like LSTM and GRU, which effectively model sequential dependencies in developed markets. However, a primary flaw in many studies is attempting to predict raw daily closing prices. Raw price movements in highly volatile emerging markets approximate a random walk and are saturated with noise. A landmark 2025 study examining 55 global stock markets, including the EGX30, demonstrated that ML algorithms using only technical indicators achieve an average accuracy of ~51%—barely exceeding random chance.

To overcome this "accuracy ceiling," recent literature advocates for a paradigm shift toward Ensemble Learning and Target Engineering. Predicting smoothed trend direction (e.g., EMA direction) rather than raw prices effectively filters out high-frequency noise. Consequently, contemporary research indicates that breaking the 51% accuracy barrier requires robust Stacking Ensembles integrated with external macroeconomic indicators.

Study	Journal / Rank	Model Architecture	Accuracy / Metric	Target Market
Livieris et al. (2025)	Springer - Q1	SVM, ANN, LSTM, GB	~51%	55 Global Markets
Park et al. (2024)	MDPI Electronics - Q2	LSTM vs. GRU	~65%	NASDAQ
Chaudhari et al. (2025)	ScienceDirect - Q2	Ensemble + Sentiment	~72%	BSE/NSE India
Springer Nature (2025)	Future Business J. - Q1	Bi-LSTM, RF, ARIMA	N/A	EGX (5 stocks)
EGX360 (This Work)	-	XGBoost + LightGBM Stack	90%	EGX30 (1998-2026)

Table 2.1: Comparison of Recent ML/DL Financial Prediction Models

2.2. Sentiment Analysis and NLP in Finance

The integration of Natural Language Processing (NLP) has transformed how markets process qualitative information. While advanced models like FinBERT successfully quantify market mood from English news streams, a critical research gap exists in Arabic-language sentiment analysis for the Egyptian market. Emerging markets suffer from a severe scarcity of structured Arabic financial datasets, and Egyptian financial discourse is often saturated with localized slang and specific economic terminologies that traditional NLP pipelines cannot interpret.

To bridge this gap, EGX360 implements a bilingual sentiment pipeline utilizing Large Language Models (LLMs). By leveraging the Llama 3.1 API, the system overcomes the lack of specialized Arabic financial lexicons through deep contextual understanding. Integrating these high-fidelity sentiment signals into the Stacking Ensemble ensures that the system captures vital localized qualitative drivers that traditional price-only models ignore.

2.3. Emerging Markets vs. Developed Markets

Unlike developed markets (e.g., S&P 500) which are characterized by high information efficiency, emerging markets like the Egyptian Exchange (EGX) exhibit significant heteroskedasticity, a low Signal-to-Noise Ratio (SNR), and "fat-tailed" return distributions. A major differentiator is their extreme sensitivity to external macroeconomic shocks. Egypt’s import-dependent economy means that equity prices are inextricably linked to the USD/EGP exchange rate, gold prices, and Central Bank of Egypt (CBE) interest rate decisions.

EGX360 addresses this gap by treating the Egyptian market as a unique ecosystem. By integrating these Egypt-specific macroeconomic features, the model captures the fundamental "fuel" that drives local market cycles. This multi-dimensional approach allows the Stacking Ensemble to achieve superior predictive accuracy (85.22%) by identifying economic regimes that traditional technical models overlook.

Market Property	Developed Markets (e.g., S&P 500)	Emerging Markets (e.g., EGX30)
Volatility	Generally lower and more predictable.	High and prone to sudden clustering.
Efficiency	High informational efficiency.	Low Signal-to-Noise Ratio (SNR).
Return Distribution	Approximates Normal Distribution.	Leptokurtic with "Fat Tails".
Main Drivers	Corporate earnings & Global growth.	Macro shocks, Currency rates & Inflation.
Participant Behavior	Dominated by Institutional investors.	High impact of Retail (emotional) investors.

Table 2.2: Comparison of Statistical and Fundamental Properties

2.4. Risk Management in Automated Trading

In high-volatility environments like the EGX, over-reliance on raw ML predictions without a robust safety net can lead to significant capital drawdown. A significant gap in current automated platforms is the lack of "contextual risk awareness," relying heavily on static, fixed-percentage exits. Prior studies highlight that a high "Down Recall"—the ability to correctly identify declining sessions—is more vital for capital preservation than overall accuracy.

EGX360 addresses these challenges through a multi-layered "Guardian Engine" that implements dynamic risk thresholds. A key innovation is the integration of the Llama 3.1 financial assistant, which analyzes the risk implications of real-time news to provide optimal decision-making support. Additionally, a high-fidelity "Paper Trading" simulator allows investors to test strategies against 28 years of historical market cycles (1998–2026) before risking actual capital, ensuring EGX360 functions as a secure financial ecosystem.

CHAPTER 3

SYSTEM DESIGN & ARCHITECTURE

3.SYSTEM DESIGN & ARCHITECTURE

3.1. Requirements (Functional & Non-Functional)

To ensure the EGX360 ecosystem effectively addresses the complexities of the Egyptian Exchange and delivers a seamless user experience, the system requirements are clearly defined into functional capabilities and non-functional constraints.

3.1.1. Functional Requirements

These requirements define the core services and complex automated tasks the system

Feature	Description
Multi-Source Ingestion	Automated scraping via reverse-engineered EGX APIs , Binance feeds, and macro scripts.
Bilingual NLP Sentiment	Arabic/English sentiment analysis of financial narratives and news using Llama 3.1 .
Technical & Macro Analytics	Real-time technical indicators (RSI, EMA) and macroeconomic driver integration.
Intelligent Forecasting	High-fidelity trend prediction (Up/Down) via optimized Stacking Ensembles .
Virtual Trading Economy	Simulated trading with encrypted virtual wallets and real-time P&L tracking.
Financial RAG Assistant	Conversational AI using RAG for contextual market insights and portfolio data.
Social Ecosystem	Real-time community feed for sharing market insights and sentiment tags .
Guardian Risk Engine	Proactive risk management with automated alerts for volatility and price-drop events.

Table 3.1: Core Functional Requirements and System Capabilities of EGX360

3.1.2. Non-Functional Requirements

These requirements define the quality attributes, performance goals, and security measures of the system:

Category	Requirement	Specification
Security	Data Privacy	Implementation of Row Level Security (RLS) within the database and full encryption for all sensitive transaction data in the virtual wallet.
Scalability	Performance	Distributed cloud infrastructure hosted on AWS and Supabase capable of handling concurrent requests and high-frequency scraping with minimal latency.
Usability	Cross-Platform	Providing a responsive and unified user experience across Android, Web, and Desktop platforms using a single Flutter codebase.
Reliability	Fault Tolerance	Ensuring 24/7 operation for Python-based daemons with automated mechanisms to handle network failures or external API rate limits.

Table 3.2: Non-Functional Requirements and System Quality Attributes.

3.1.3. System Use Case Diagram

The EGX360 Use Case Diagram defines the primary interactions between the users and the system's intelligent background services:

- **Retail Investor Operations:** Managing profiles, viewing real-time market data/indicators, executing paper trades, and engaging in the social community feed.
- **AI & Automation Daemons:** Handling autonomous scraping, bilingual sentiment analysis (Llama 3.1), and generating market trend forecasts via Stacking Ensembles.
- **Intelligent Risk & Assistance:** Managing alert thresholds via the **Guardian Engine** and interacting with the **RAG-powered AI Financial Assistant** for contextual support.

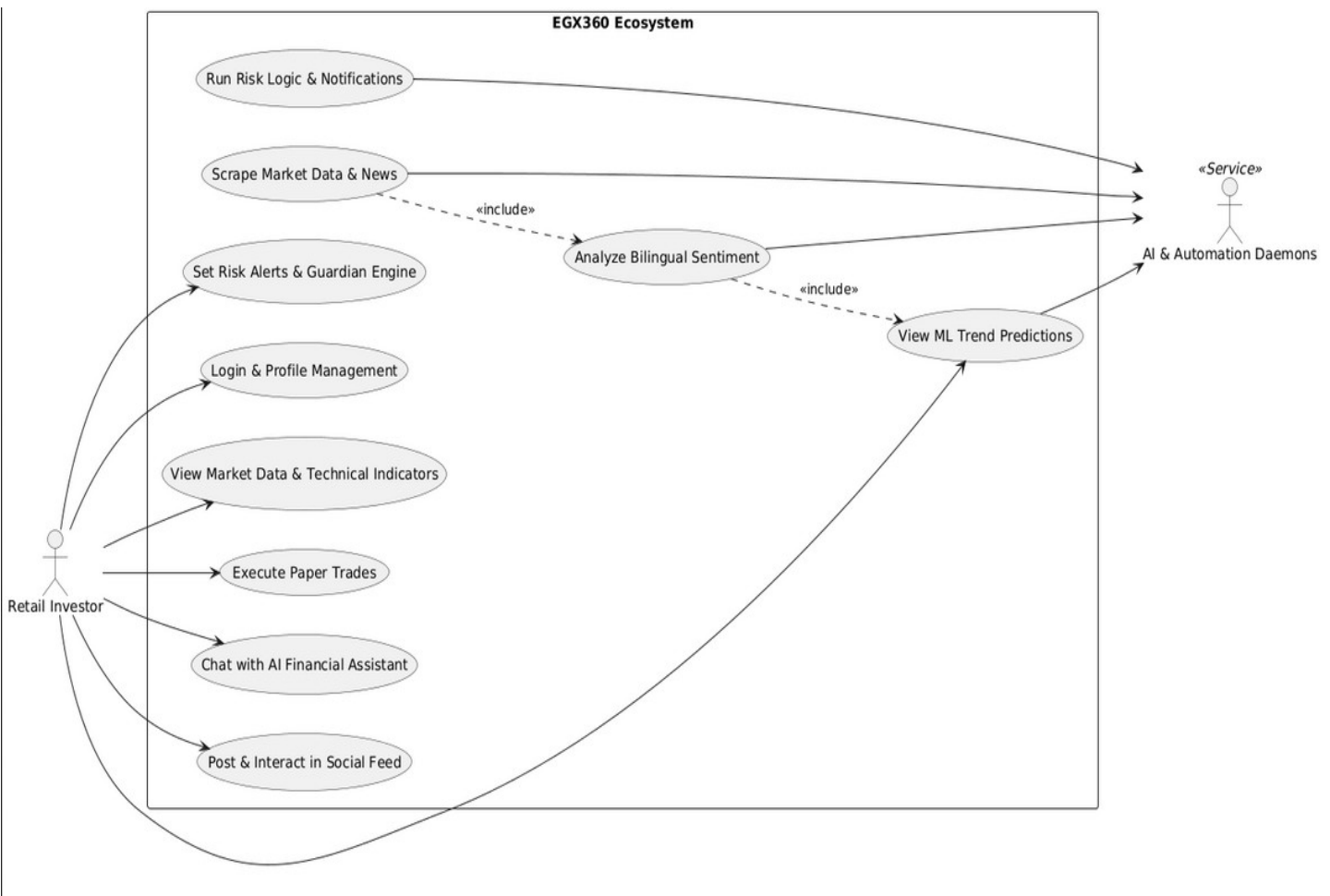


FIGURE 3.1.3: USE CASE DIAGRAM REPRESENTING USER INTERACTIONS AND BACKGROUND SERVICE AUTOMATION.

3.2. High-Level Architecture Diagram

The **EGX360** ecosystem utilizes a distributed, **4-tier architecture** designed to bridge real-time financial data with machine learning intelligence.

- **Raw Data Sources:** Ingesting real-time market data via reverse-engineered EGX endpoints, Binance APIs, and macroeconomic scripts.
- **Intelligence & Automation (AWS EC2):** Hosting the Python-based daemons for bilingual sentiment analysis and trend forecasting using Stacking Ensembles (XGBoost/LightGBM).
- **Persistence & Security (Supabase):** Managing centralized PostgreSQL storage with **(RLS)** for data privacy and real-time database synchronization.
- **Delivery & Presentation (Flutter):** Distributing real-time financial insights and FCM-based alerts to the multi-platform application developed using Flutter.

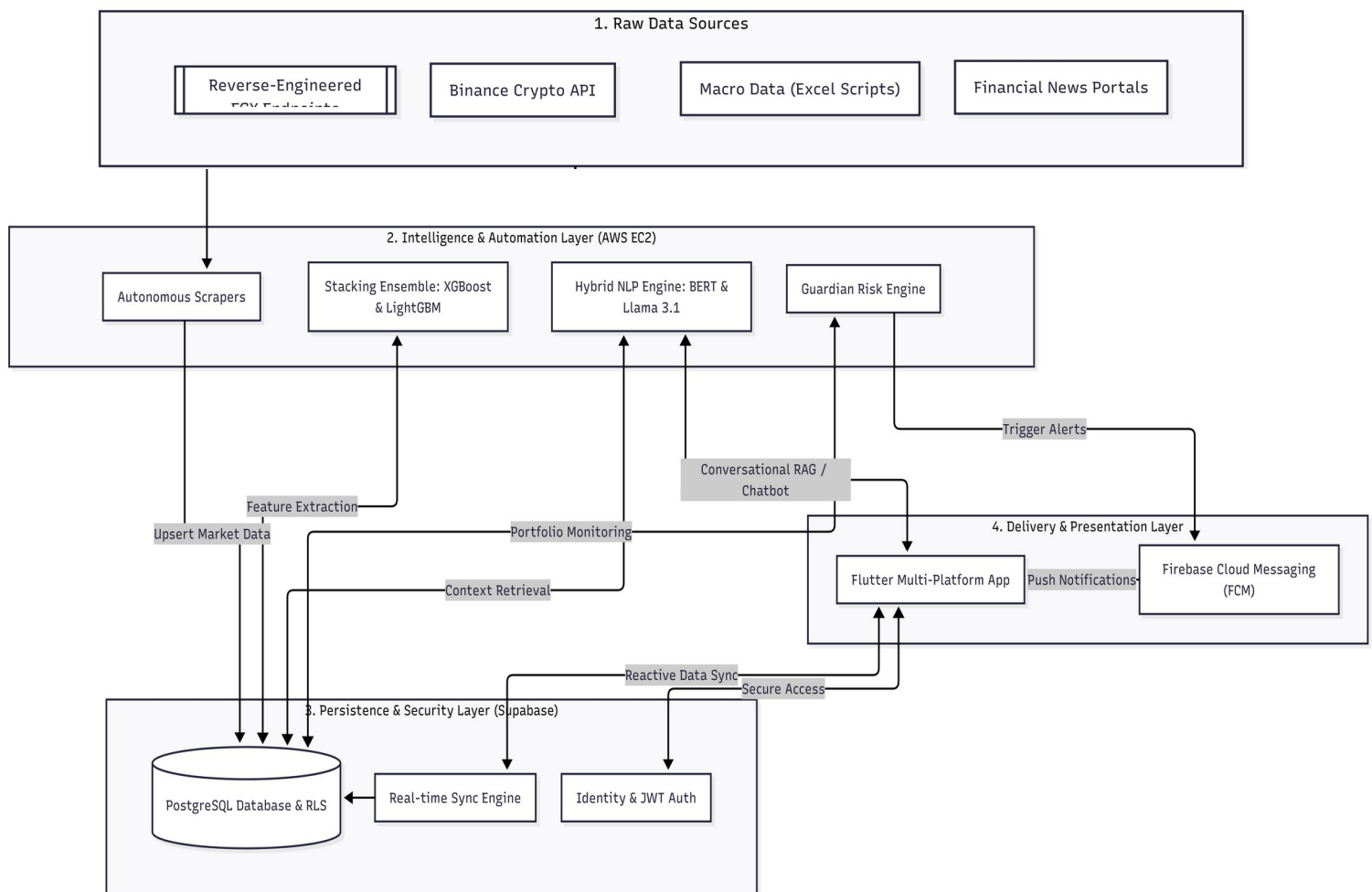


FIGURE 3.2: HIGH-LEVEL 4-TIER ARCHITECTURE.

3.3. Database Schema and Conceptual Model

designed to handle fast market updates and social interactions while keeping user data private and secure. The data is organized into three main areas:

- **User & Social Area:** Manages user profiles and the community hub, including posts, comments, and the follower system in both Arabic and English.
- **Market & AI Insights:** Connects live stock prices with AI-driven trend predictions and automated news analysis.
- **Trading & Risk Management:** Controls the virtual trading engine, including wallets, portfolio tracking, and **Guardian Engine** for automatic risk protection.

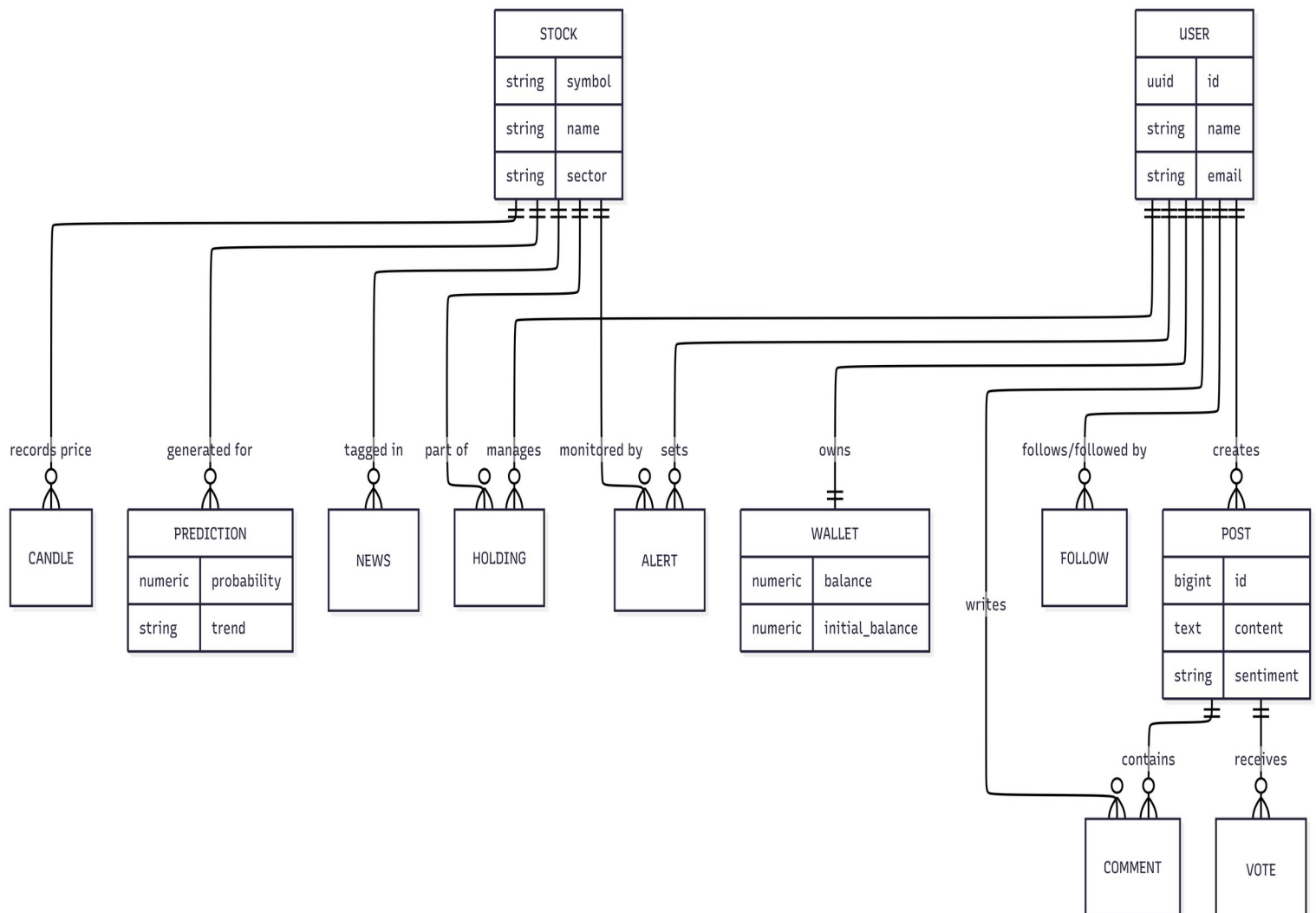


Figure 3.3: Conceptual Entity-Relationship Diagram (ERD).

3.4. Sequence Diagrams

3.4.1. Autonomous News Ingestion & Cerebras AI Analysis

This sequence illustrates the integrated pipeline for financial data ingestion, hybrid sentiment analysis, and market trend forecasting using Stacking Ensembles.

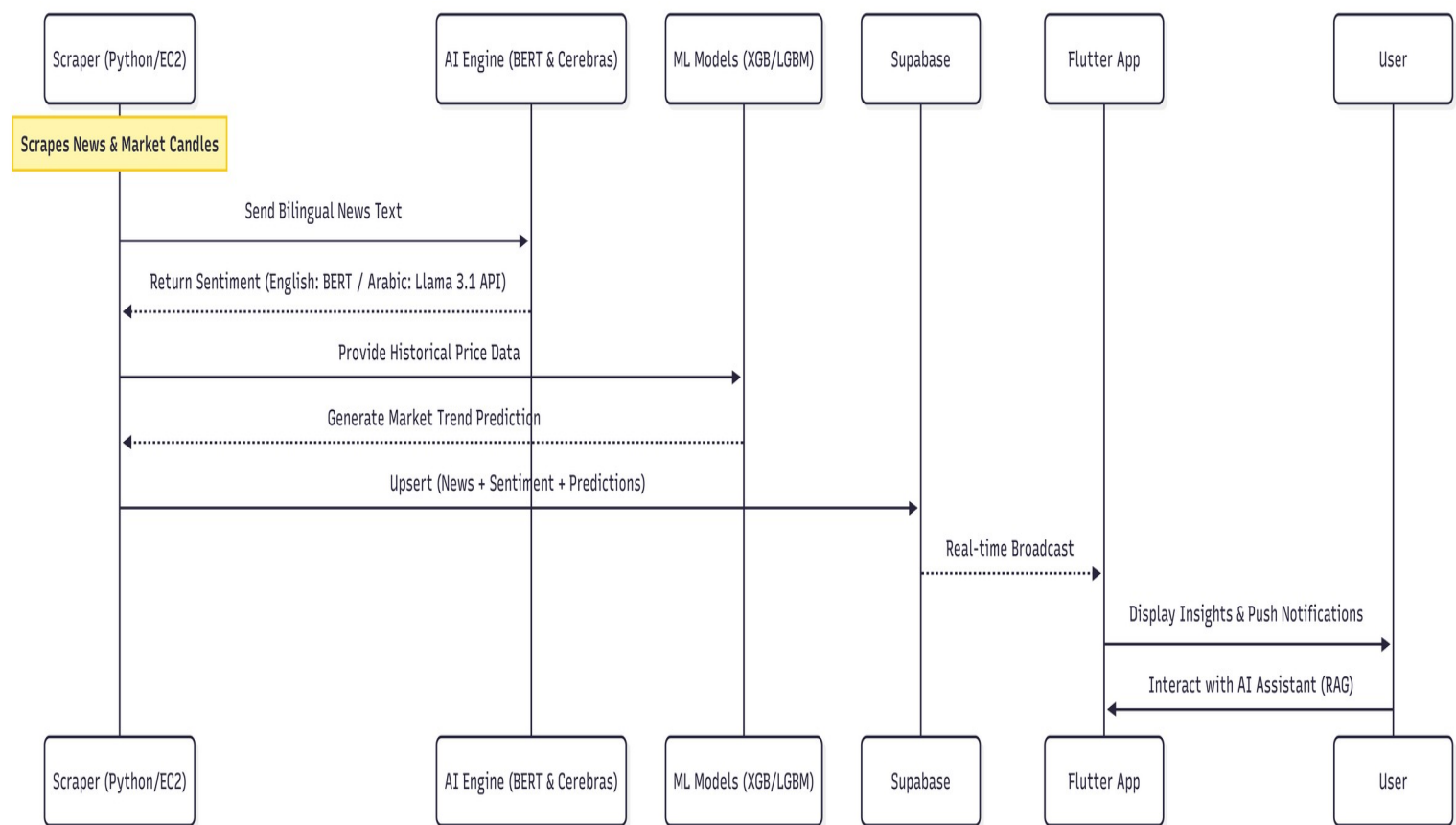


Figure 3.4.1: Sequence Diagram of the Automated AI News and Market Analysis Pipeline.

3.4.2. Guardian Engine & Risk Management Sequence

This sequence details the automated risk management flow where the background engine monitors real-time portfolio health against user-defined protection rules (e.g., stop-loss thresholds) and dispatches real-time alerts.

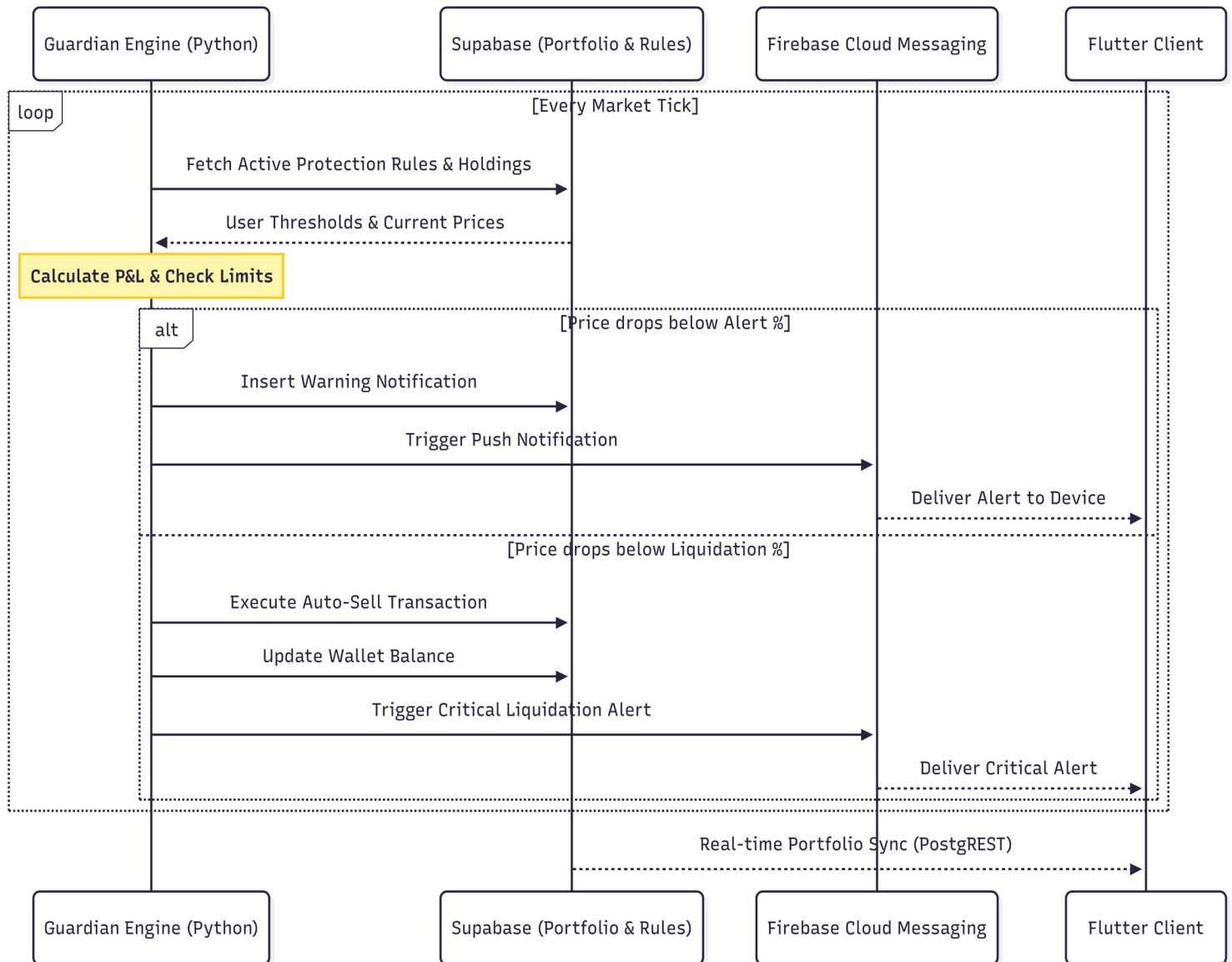


Figure 3.4.2: Sequence Diagram of the Guardian Engine's Automated Risk Mitigation Flow.

3.4.3. Virtual Trade Execution Sequence

This sequence illustrates the transactional logic of the paper-trading engine. It details how the Flutter client interacts with Supabase Remote Procedure Calls (RPCs) to validate wallet balances, calculate average holding prices, and securely record transactions.

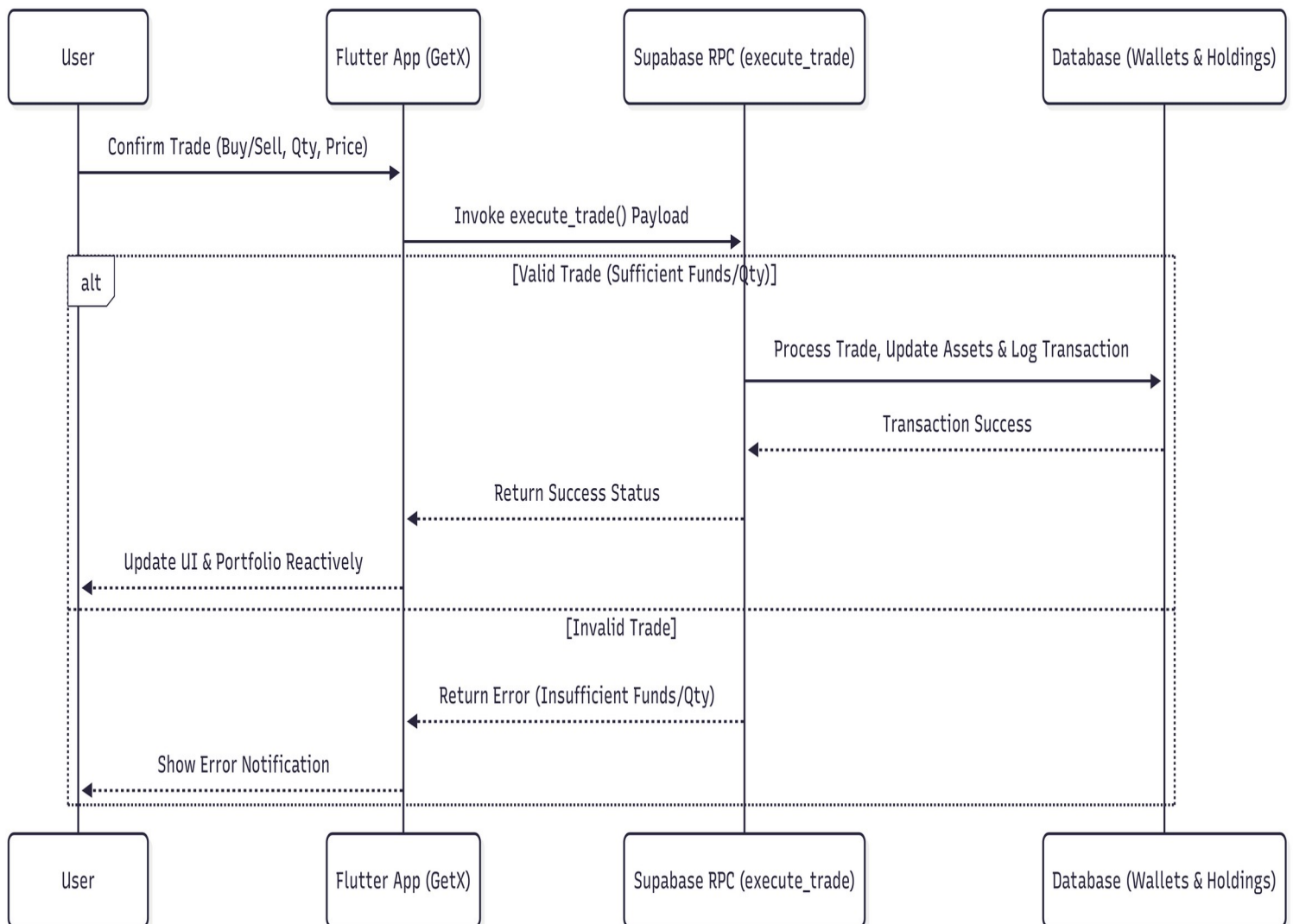


Figure 3.4.3: Sequence Diagram of the Virtual Trade Execution and Transactional Logic.

3.5. Comprehensive Service-Level Architecture

This diagram illustrates the core service-level architecture of **EGX360**, highlighting the integration of user management, virtual trading, background risk assessment, and the AI pipelines.

- **VirtualTradingService:** Manages paper-trading business logic, fund validation, and P&L calculations prior to database execution.
- **GuardianEngineService:** An autonomous background worker that continuously monitors portfolio risk and triggers protective measures (e.g., auto-selling).
- **AIPredictionService:** Handles tabular data forecasting and market trend analysis using XGBoost and LightGBM stacking ensembles.

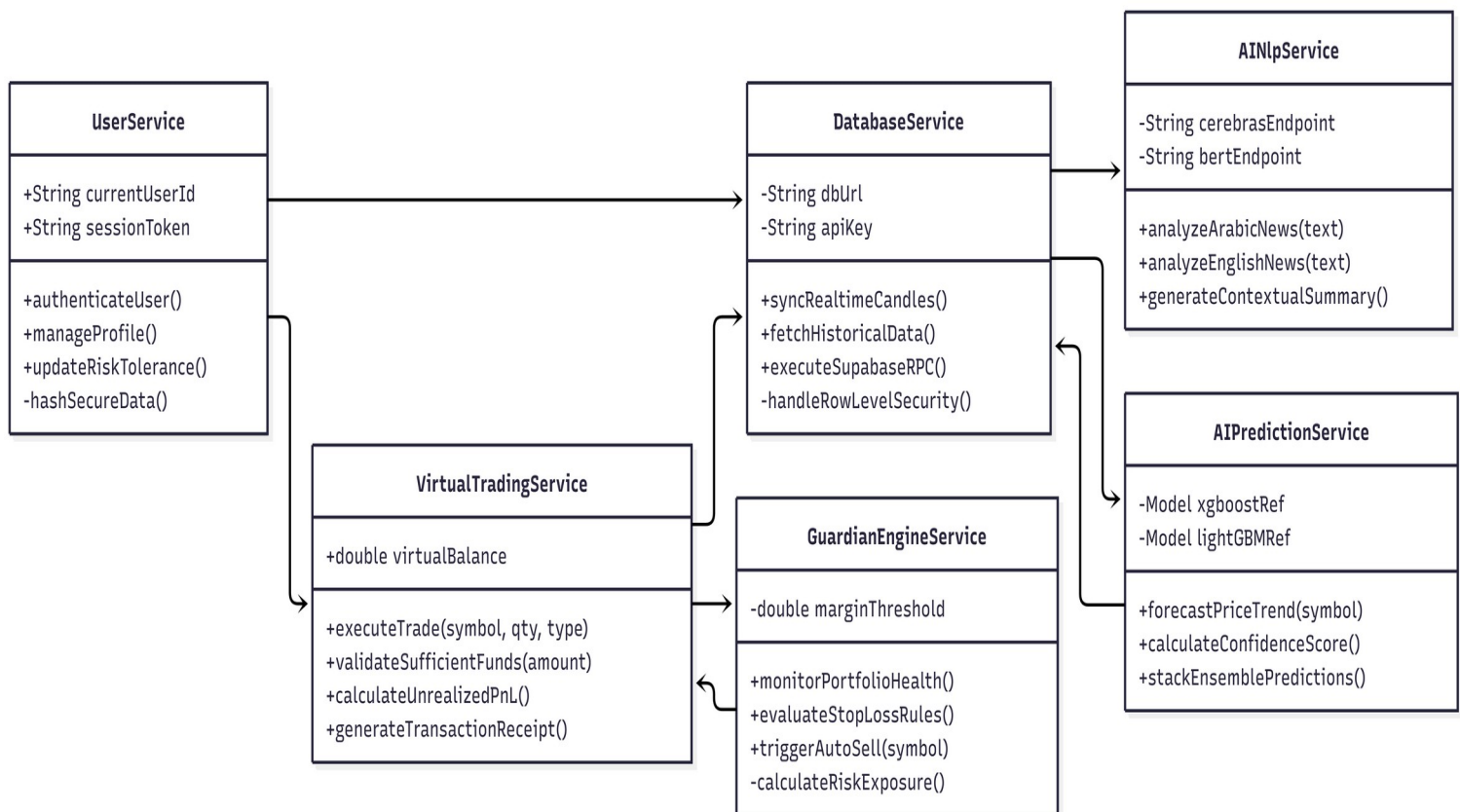


Figure 3.5: Comprehensive Service-Level Class Diagram demonstrating extended feature encapsulation and module interactions.

3.6. Activity Diagram

3.6.1. Activity Diagram: App Initialization & Navigation Hub

This diagram focuses on the critical startup sequence, including authentication guards, personalized onboarding, and the primary navigation hub using the Bottom Navigation Bar.

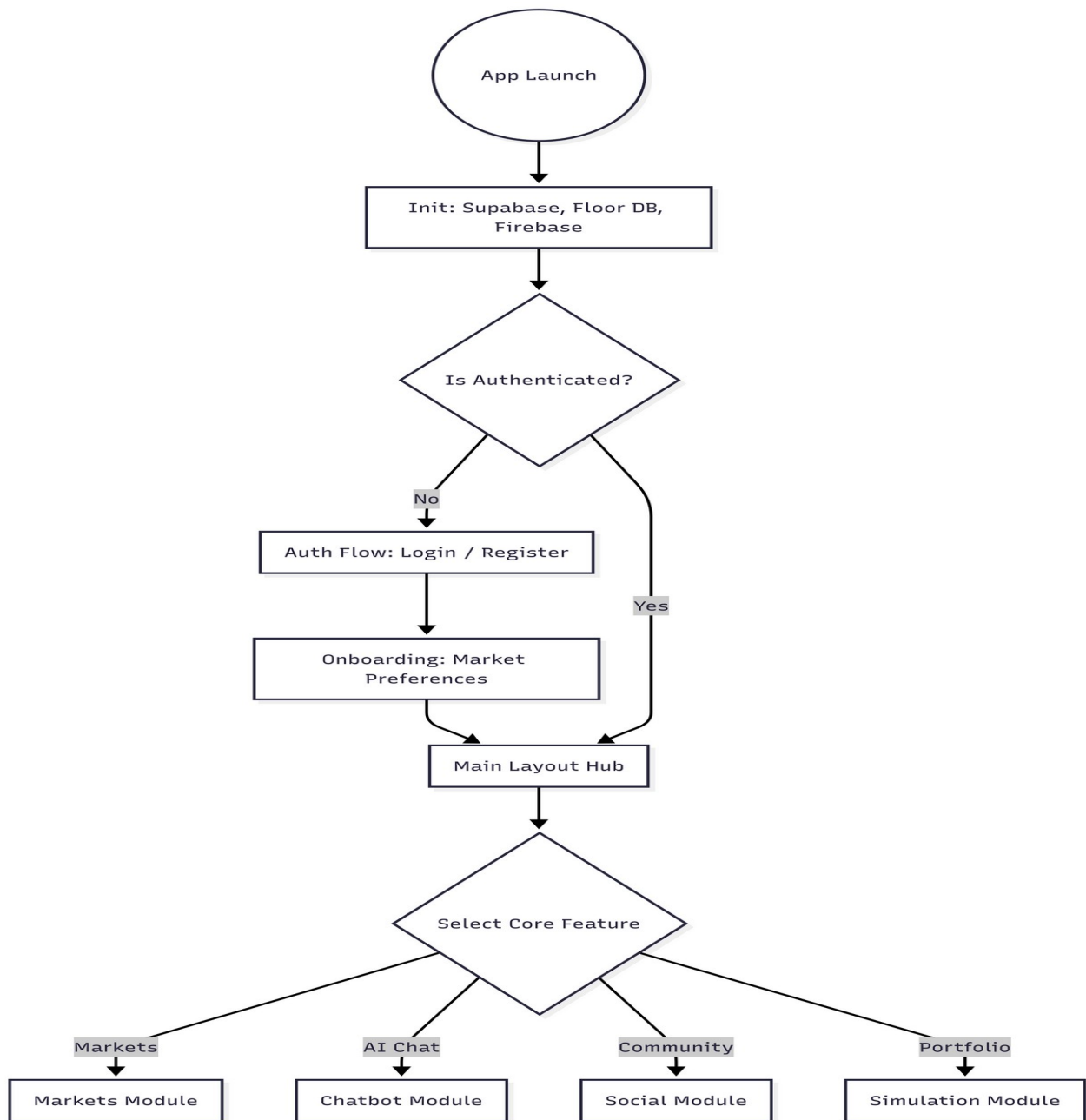


Figure 3.6.1: Activity Diagram illustrating the application's startup sequence.

3.6.2. Activity Diagram: Core Feature Workflows

This diagram provides a detailed breakdown of the internal logic for the primary modules within **EGX360**. It illustrates how user actions are processed within the Markets, AI Chatbot, and Community features.

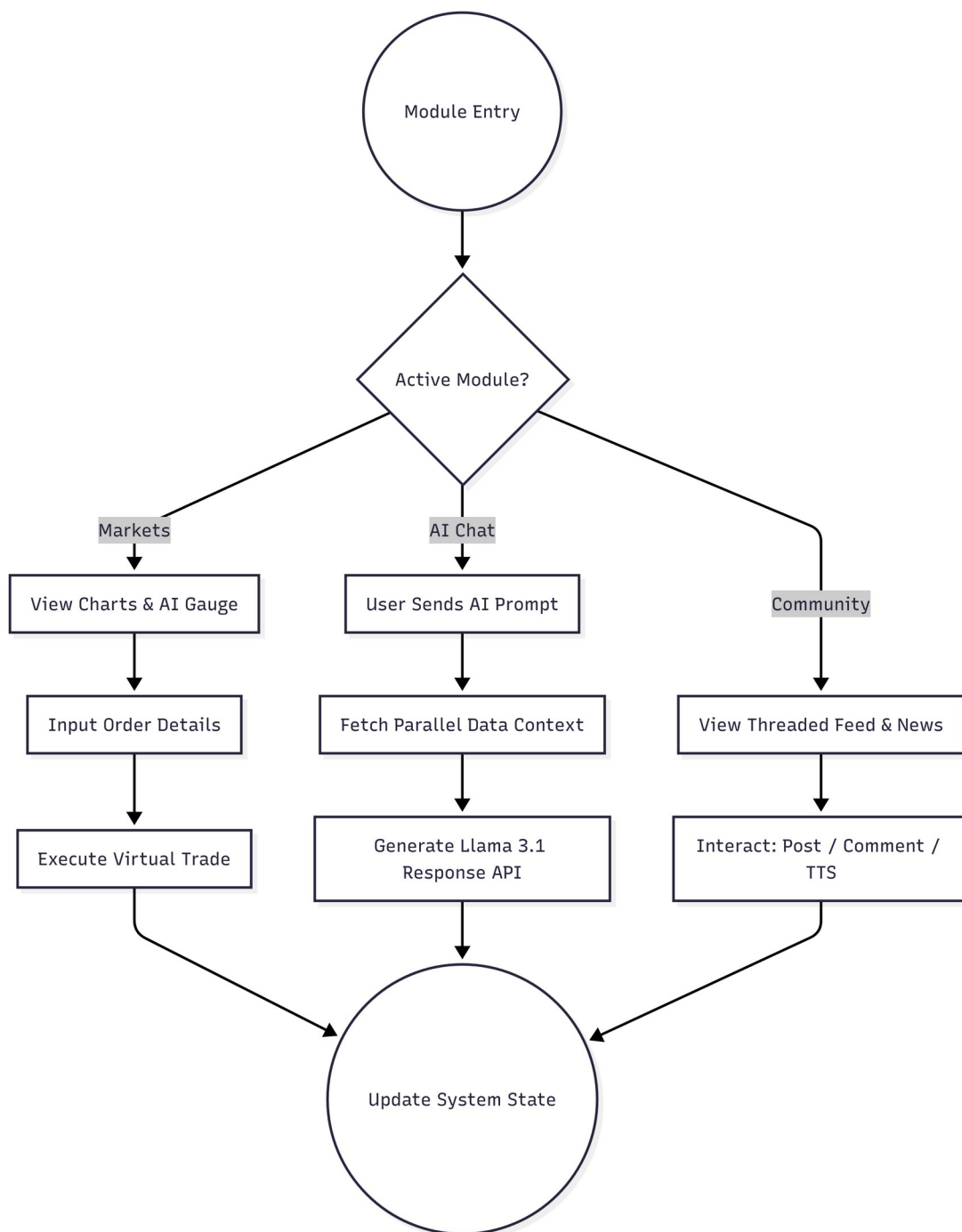


Figure 3.6.2: Activity Diagram illustrating the operational logic within EGX360's core feature modules.

3.7. Technology Stack Selection

The EGX360 ecosystem utilizes a modern, decoupled technology stack selected for its scalability, real-time capabilities, and support for complex Arabic NLP and financial modeling.

Category	Technology	Reason for Selection
Multi-Platform App	Flutter & Dart	Enables a unified, high-performance experience across Android, Web, and Desktop from a single codebase.
Backend & Database	Supabase (PostgreSQL)	Provides real-time synchronization, centralized authentication, and Row Level Security (RLS) for sensitive financial data.
Language	Python	The industry standard for financial scraping, data science, and integrating advanced ML models.
Cloud Hosting	AWS (EC2)	Scalable infrastructure required to host persistent Python daemons for market scraping and AI inference.
AI (Forecasting)	XGBoost / LightGBM	Optimized Stacking Ensembles designed for high-fidelity tabular data trend prediction.
AI (NLP & RAG)	Llama 3.1 API / BERT	Advanced bilingual support (Arabic/English) for sentiment analysis and contextual financial assistance.
DevOps & Deployment	Docker	Utilized to containerize the Python daemons, ensuring consistency between development and production environments on AWS .

Table 4.1: EGX360 Technology Stack and Selection Rationale

CHAPTER 4

BACKEND INFRASTRUCTURE & DATA ENGINEERING

4.BACKEND INFRASTRUCTURE & DATA ENGINEERING

Reliable forecasting requires high-quality, low-latency data. To ensure data integrity and real-time availability, the backend ingestion layer is designed as an autonomous pipeline that collects, validates, and streams financial data into the system.

The pipeline performs three core functions:

- **Real-time Data Acquisition** from multiple sources.
- **Data Cleaning & Validation** to eliminate anomalies.
- **Efficient Data Streaming** into Supabase for model inference and trading simulation.

4.1. Data Sourcing & Scraper Engineering

Due to the absence of a public EGX API, a custom multi-threaded scraping architecture was developed to supply real-time and historical data required for both AI models and the paper-trading engine.

4.1.1. EGX Data via Reverse Engineering

This pipeline supplies the high-frequency data (1m and 1d candles) required for paper-trading and ML training. It interfaces with external WebSockets via a multi-threaded Python daemon running on AWS EC2.

A. Daemon Lifecycle & Execution Flow

The background service operates as a state-driven daemon:

1. **Bootstrap (Initial Catch-up):** On startup, it fetches the last 300 minutes (1m) and 5 days (1d) of candles to prevent data gaps from system downtime.
2. **Active Market Loop:** The service continuously queries Cairo local time. During Market Open, it fetches the latest data, filters it, and upserts to Supabase at fixed intervals (e.g., every 20 seconds). During Market Closed, it executes a "Final Sync" (last 100 bars) to guarantee session integrity, then enters an idle state.

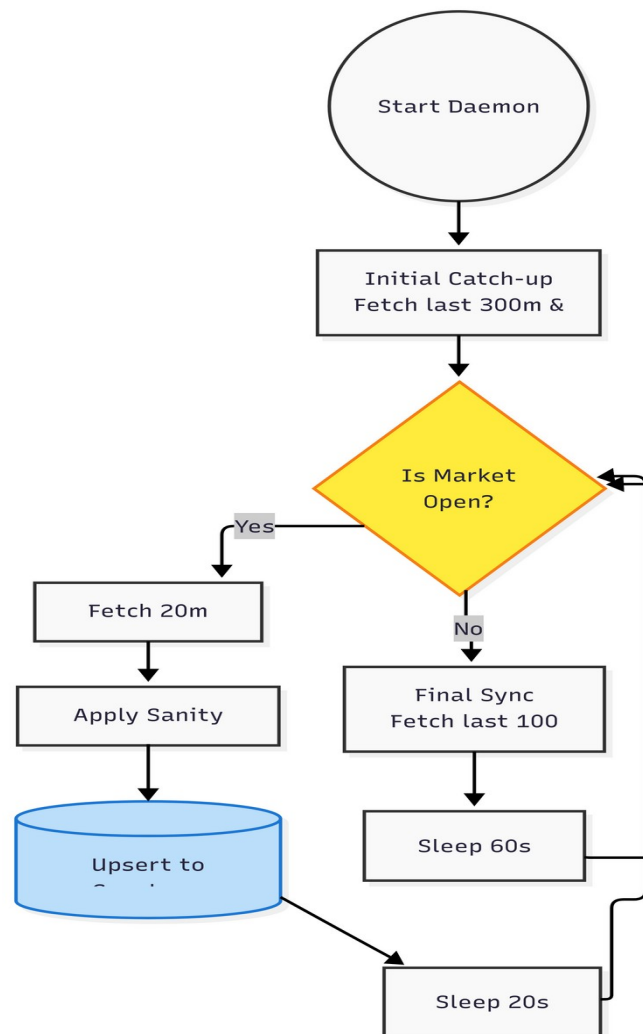


Figure 4.1: State-driven execution loop of the EGX background daemon.

B. Multi-Threaded Ingestion & Proxy Load-Balancing

To handle multiple symbols efficiently, the system utilizes multi-threaded parallelism, where each thread is responsible for a specific symbol. This prevents blocking and ensures continuous data flow. Additionally, proxy-based routing is used to distribute requests and avoid rate limiting.

C. Technical Connection & WebSocket Handshake

The system maintains persistent WebSocket connections for real-time data streaming.

- **Session Management:** Each connection initializes a session for symbol tracking and streaming.
- **Data Streaming:** The daemon continuously listens for incoming price updates and processes them in near real-time.

D. Data Cleaning & Sanity Filters

Before database insertion, raw data is validated through the following programmatic filters:

Feature/Check	Logic / Constraint	Technical Purpose
Market Hours	10:00 AM to 02:45 PM (Cairo Time)	Ignores out-of-market test prints and administrative adjustments.
Volume Check	if volume ≤ 0	Drops candles with zero trading activity.
Price Logic	if low $>$ high or price ≤ 0	Discards corrupt packets or mathematical anomalies.
Duplicate Prevention	PostgreSQL Upsert	Updates existing records by timestamp instead of creating duplicates.

E. Persistence Layer

Validated records are formatted into JSON and sent to Supabase using a PostgreSQL Upsert operation. If network latency causes a duplicate submission, the database updates the existing row instead of throwing a primary key violation.

4.1.2. Real-Time Crypto Streaming (Non-Persistent)

To provide global financial context, the system integrates real-time cryptocurrency feeds (e.g., BTC/USD).

A. Technical Stream Architecture

The ingestion service utilizes a Python background client connecting directly to the

- **Socket Subscription:** The client subscribes to raw aggregate trade streams which broadcast tick-by-tick market executions.
- **Non-Persistent Data Flow:** To prevent database resource exhaustion, high-frequency cryptocurrency ticks are processed in-memory and broadcasted directly to clients, bypassing database storage entirely.

4.1.3. Local Commodities & Market Summary Scraper

This pipeline gathers broad market statistics and retail gold prices.

A. EGX Market Summary Scraper

This script retrieves the daily total market cap and transaction value.

- **Anti-Bot Bypass:** Uses headless browser automation to effectively bypass the portal's firewall and verification checks.
- **Data Extraction & Formatting:** Queries the Document Object Model (DOM) to isolate specific numeric values, utilizing Regular Expressions to strip currency symbols and normalize the scale.
- **Storage Logic:** Queries the database for the current trade date; updating the record if it exists, or inserting a new daily row.

B. Gold Price Scraper & Volatility Trigger

Tracks retail gold prices and flags market volatility

1. **Cloudflare Bypass & Parsing:** The target source is protected by Cloudflare. The script uses specialized bypassing libraries to handle JavaScript challenges and maps Arabic table labels to English database keys.
- **Volatility Event Trigger:** Before database insertion, the engine fetches the last recorded 21K buy price and calculates the absolute difference against the newly scraped price:

$$\Delta = |P_{\text{new}} - P_{\text{old}}|$$

If the difference breaches a predefined threshold($\Delta \geq 5.0$ EGP), the script dispatches a programmatic trigger to the notification service, decoupling the scraping loop from alert execution.

- **Storage Logic:** Upserts the parsed pricing dictionary mapped with Cairo timezone metadata into the database.

4.1.4. Autonomous News Scraping & Anti-Bot Strategies

To handle diverse and dynamic news sources, the system implements a multi-layered scraping strategy:

- **A. Dynamic Source Handling:** RSS feeds and redirected links are resolved automatically.
- **Anti-Bot Handling:** The system employs adaptive request strategies to avoid blocking during large-scale scraping.
- **Tiered Extraction Pipeline:**
 1. Generic HTML parsing for standard pages.
 2. Domain-specific parsers for structured sources.
 3. Final text cleansing to remove noise.

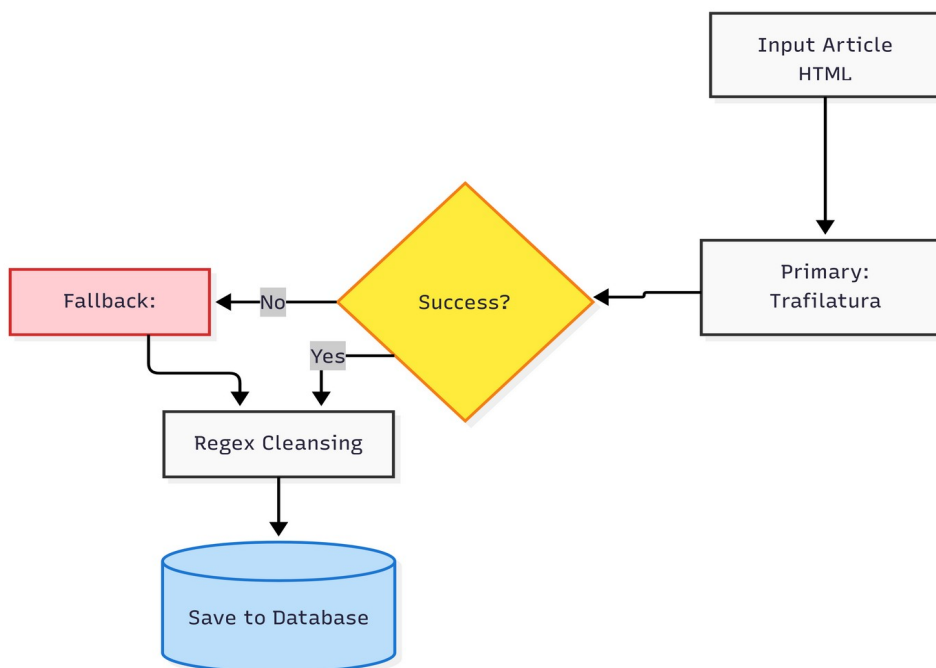


Figure 4.2: The three-tier fallback mechanism for news content extraction.

- **Heuristic De-duplication:** Prevents database pollution via cross-referencing. Drops URLs originating from social media platforms. Compares new titles against the database using fuzzy string matching. If the similarity ratio exceeds 80%, the article is flagged as a duplicate and dropped.

4.2. Data Processing & NLP Pipeline

Once raw news articles are collected, they are processed through a structured NLP pipeline designed to clean, normalize, and extract meaningful financial sentiment signals. The pipeline is optimized for bilingual processing (Arabic and English) and integrates both Large Language Models (LLMs) and specialized financial NLP models.

4.2.1. Text Cleaning & Metadata Extraction

Before applying machine learning models, the system performs lightweight preprocessing to ensure data quality and consistency:

- **Language Detection & Routing:** Articles are automatically classified as Arabic or English and routed to the appropriate processing pipeline.
- **Context Optimization:** Article titles are combined with their content to improve contextual understanding, while input length is controlled to ensure efficient processing without exceeding model limits.

4.2.2. Arabic Sentiment Analysis (Llama 3.1 via Cerebras API)

To process Arabic news efficiently, the system queries the remote Cerebras Cloud API using Meta's Llama 3.1 (8B) model. To minimize API latency, a single engineered prompt executes three simultaneous tasks:

- **Validation:** Returns a strict boolean to discard corrupted HTML strings.
- **Sentiment:** Assigns a definitive market label (Positive, Negative, or Neutral).
- **Cleansing:** Rewrites the exact text while stripping embedded ads without summarization.
- **Execution Constraints:** The request enforces low temperature settings to guarantee deterministic, non-creative outputs.

- **Parsing Logic:** The LLM outputs data within strict custom tags, which are parsed locally using Regex before database insertion.

4.2.3. English Sentiment Analysis (FinBERT)

This multi-task approach reduces processing overhead while improving the overall quality of extracted signals. To ensure consistency and reliability, the model operates under controlled generation settings and produces structured outputs that can be directly integrated into the data pipeline.

- **Efficient Local Execution:** The model runs locally to eliminate external API latency and significantly reduce operational costs.
- **Probability Mapping:** It evaluates the text and outputs a direct mathematical sentiment probability, which is subsequently mapped to the final database label.

4.3. Database Architecture & Security

The system utilizes a relational database architecture designed to support high-frequency financial data, real-time interactions, and secure user transactions. The design emphasizes data integrity, scalability, and strict access control to support both analytical workloads and user-facing features.

4.3.1. Modular Entity-Relationship Diagram (ERD) Strategy

Due to the system's complexity, the database follows a Domain-Driven Design (DDD) approach, where the schema is divided into three core modules:

1. Market Data & Intelligence Core

This module handles high-frequency market data, AI predictions, and NLP-processed news, forming the analytical backbone of the system.

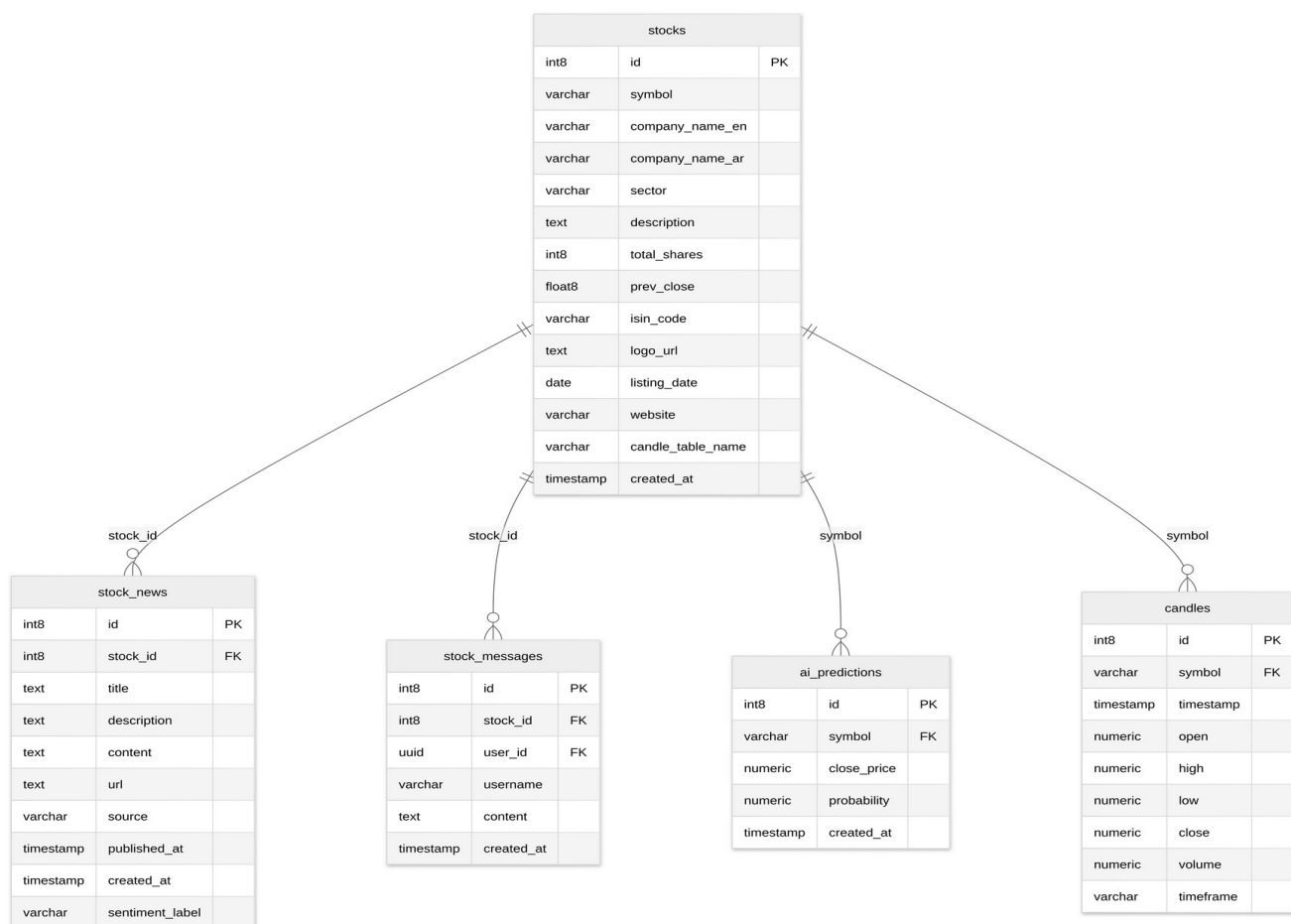


Figure 4.3.1a: ERD of the Market Data & Intelligence Core.

2. Portfolio & Paper-Trading Engine

This module manages user financial states, including virtual wallets, holdings, and transaction history. It enforces strict relational constraints to ensure data consistency and accurate portfolio tracking.

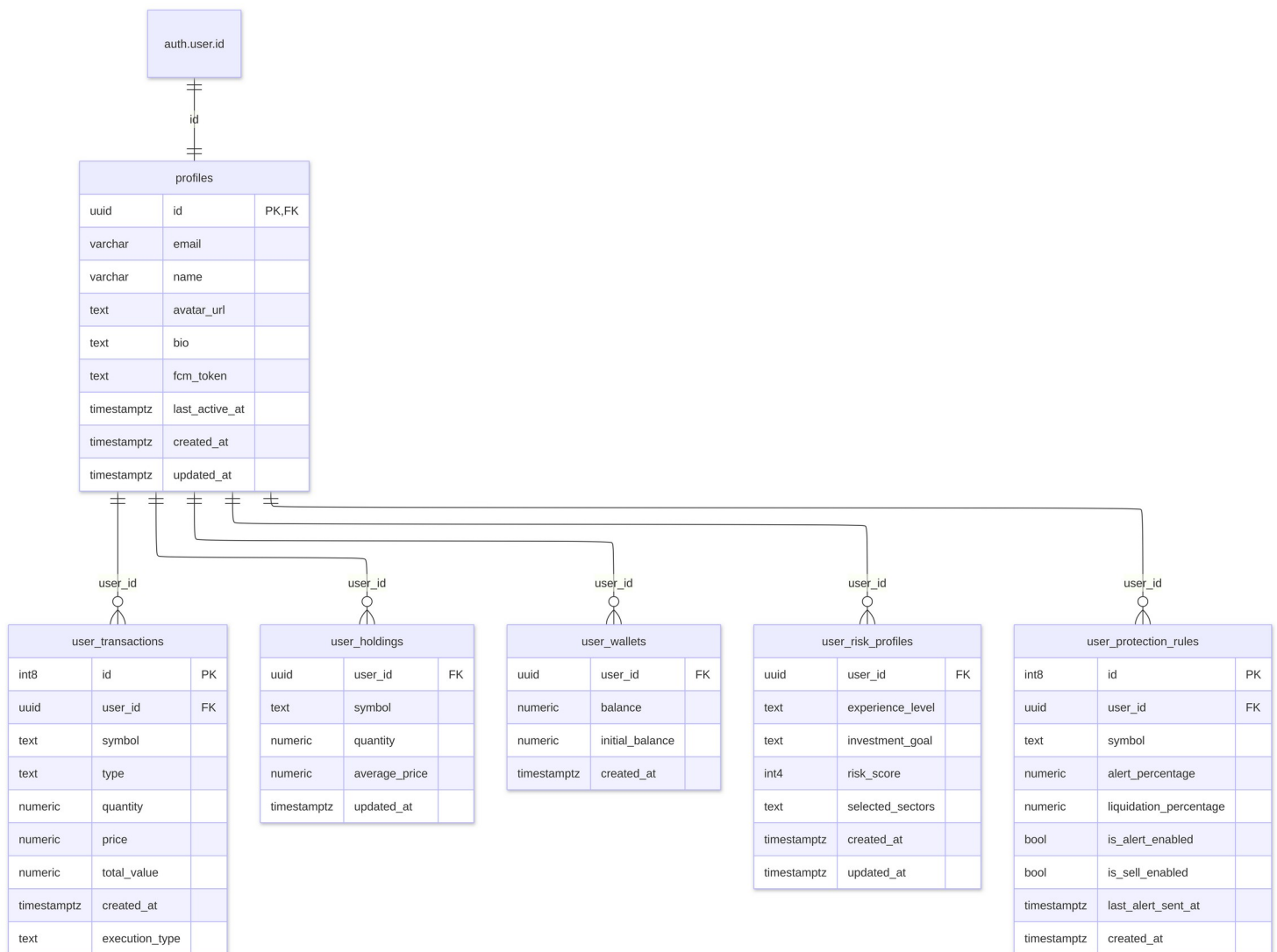


Figure 4.3.1b: ERD of the Portfolio & Paper-Trading Engine

1. 3. Social & Community Layer

This module manages user profiles, social interactions, and engagement features such as posts, comments, and follow relationships.

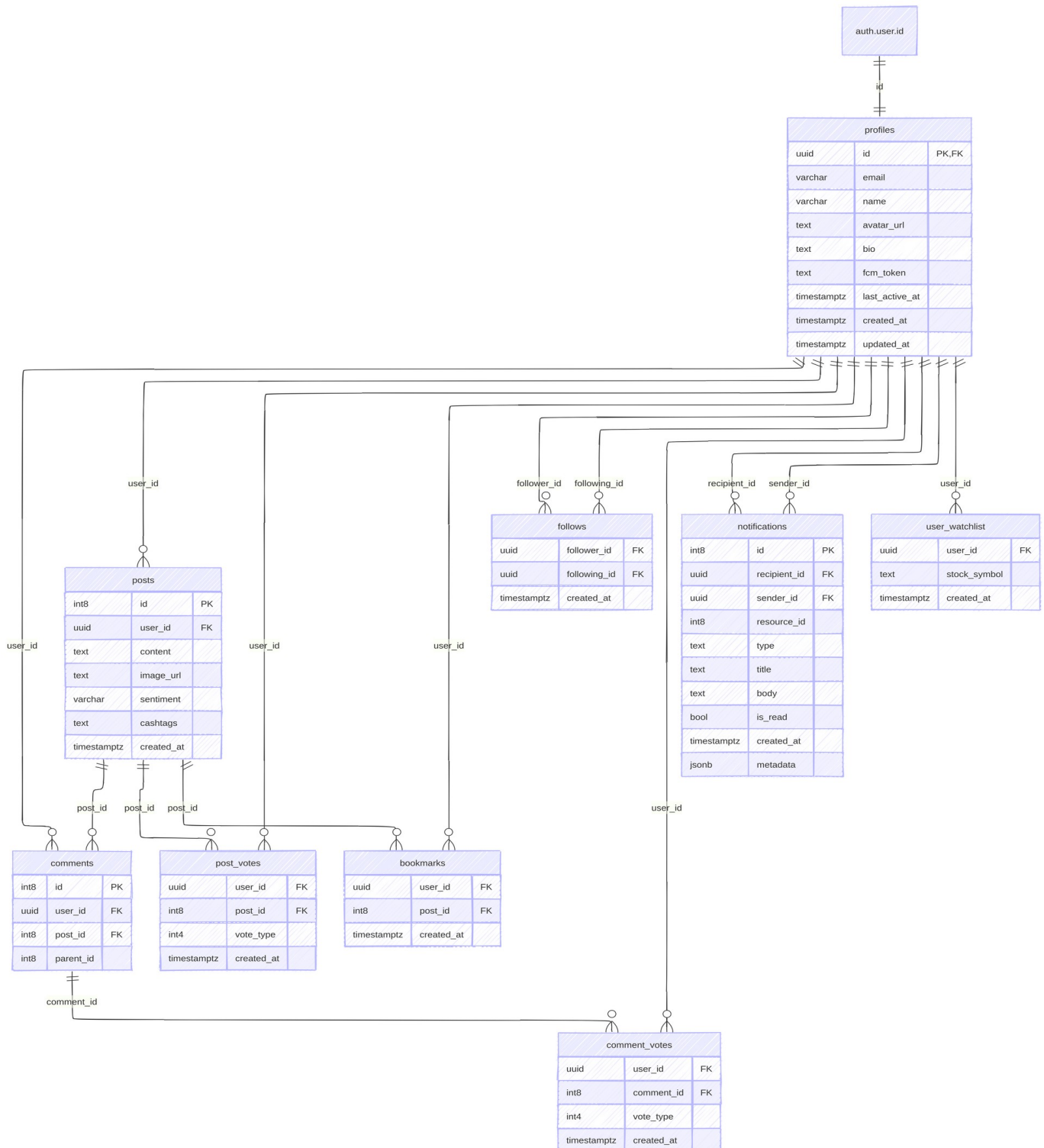


Figure 4.3.1c: ERD of the Social & Community Layer

4.3.2. Database Optimization (Normalization, Indexes, & Triggers)

To ensure high performance under heavy workloads, several optimization strategies are applied:

A. Relational Normalization

Designed in the Third Normal Form (3NF). Static entities (e.g., market metadata, user profiles) are strictly isolated from dynamic transactional entities to eliminate redundancy and minimize the storage footprint.

B. Performance Indexing

To accelerate query execution on filters, joins, and sorting operations, targeted indexes are applied:

Index Name	Target Table	Target Column(s)	Index Type / Purpose
idx_posts_cashtags	posts	cashtags	GIN Index for fast full-text array searching.
idx_posts_user_id	posts	user_id	B-Tree for instantaneous user profile feed rendering.
idx_comments_post_id	comments	post_id	B-Tree to speed up nested comment threads loading.
idx_notifications_recipient	notifications	recipient_id	B-Tree for immediate alert fetching.
idx_notifications_recip_read	notifications	recipient_id, is_read	Composite Index to query unread notifications first.
idx_ai_predictions_created_at	ai_predictions	created_at DESC	Descending Index for retrieving the latest predictions instantly.

C. Automated Database Triggers

- **Automated Triggers:** Database-level triggers are used to enforce business logic and maintain data integrity without relying on application code.

4.3.3. Secure Virtual Wallet & Transaction Ledger

The trading system follows a ledger-based design to ensure financial integrity and prevent inconsistencies.

- **Immutable Transactions:** All operations are recorded in an append-only transaction log instead of directly modifying balances.
- **Atomic Execution:** Each trade is executed as a single atomic transaction, ensuring that balance updates and holdings are always consistent.
- **Average Price Calculation:** The weighted average cost is mathematically recalculated during each purchase:

$$\text{New Avg Price} = ((Q_{\text{current}} \times P_{\text{current}}) + (Q_{\text{new}} \times P_{\text{new}})) / (Q_{\text{current}} + Q_{\text{new}})$$

- **Concurrency Control:** Database-level locking mechanisms prevent race conditions and ensure safe execution under concurrent requests.

4.3.4. Security & Row Level Security (RLS) & Policies

To protect sensitive financial data, the system enforces strict Row Level Security (RLS) policies at the database level. This ensures that users can only access their own financial data, public data remains accessible where required, and unauthorized operations are blocked even if API keys are exposed:

Table Name	RLS Status	Read Access (SELECT)	Write Access (INSERT/UPDATE/DELETE)
stocks, stock_news, materials_prices, market_history	Disabled	Public (Anonymous Read)	Blocked for all clients (Admin Only)
profiles	Enabled	Public (True)	Owner Only (auth.uid() = id)
posts, comments, stock_messages	Enabled	Public (True)	Owner Only (auth.uid() = user_id)
user_wallets, user_holdings, user_transactions, user_protection_rules	Enabled	Owner Only	Owner Only (auth.uid() = user_id)
notifications, bookmarks, user_watchlist	Enabled	Owner Only	Owner Only (auth.uid() = user_id)
ai_predictions	Enabled	Public (True)	Public Insert (ai_pred_public_insert)

4.3.5. Supabase RPCs (Advanced Analytics)

To reduce latency and minimize client-server communication, critical business logic is executed directly at the database layer using Remote Procedure Calls (RPCs).

This approach provides:

- **Reduced network overhead:** Consolidates multiple client requests into a single database call.

- **Faster execution:** Runs compiled SQL statements directly on the server host.
- **Stronger data consistency:** Enforces transactional boundaries at the storage engine level.

Key RPCs include:

- **execute_trade:** Executes trades atomically with balance validation.
- **setup_user_portfolio:** Initializes user portfolios based on risk preferences.
- **get_posts_with_status:** Aggregates social data in a single query.
- **get_posts_with_status:** Aggregates social data in a single query.

4.4. Authentication & User Access Control

The system relies on Supabase Auth (powered by GoTrue) to deliver a secure, scalable, and frictionless authentication experience.

4.4.1. OAuth 2.0 & Social Integration

The platform supports standard OAuth 2.0 flows (e.g., Google Authentication) alongside traditional email/password methods. Delegating authentication to major identity providers reduces onboarding friction and enhances overall system security.

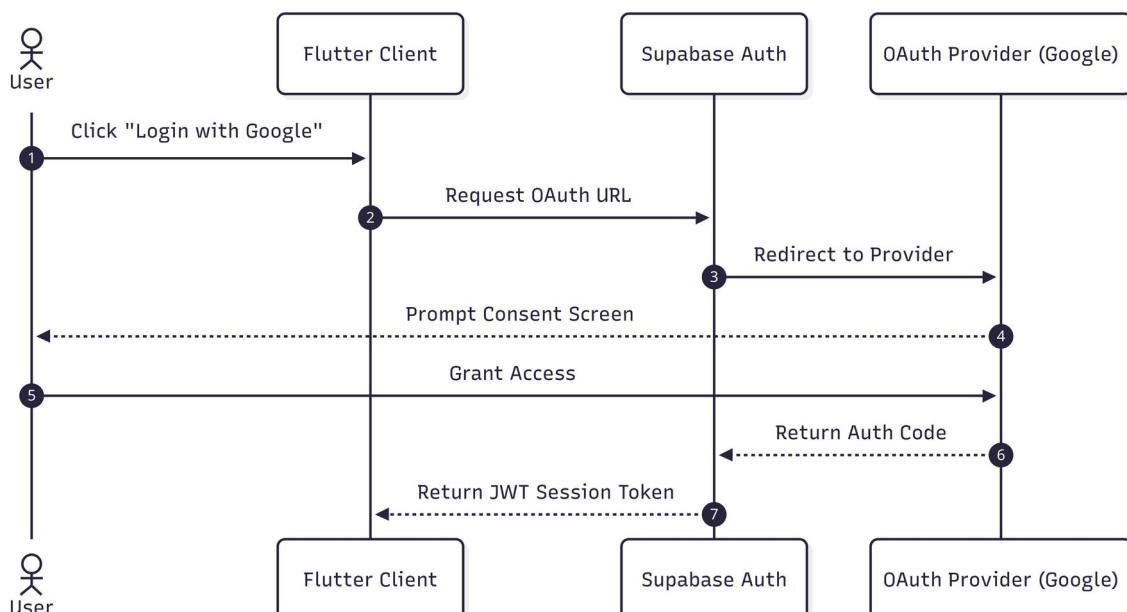


Figure 4.4: Sequence diagram illustrating the OAuth 2.0 authentication flow.

4.4.2. Email Verification & Deep Linking

The system integrates secure email verification using One-Time Passwords (OTPs) and Magic Links delivered via an SMTP relay. To enhance user experience, deep-linking is implemented, allowing authentication links to open directly within the application without requiring manual code input. This ensures a seamless and secure verification process across devices.

4.4.3. Secure User State Management

Maintaining session security on the client-side is critical to prevent session hijacking or unauthorized data access. The application manages state using strict token lifecycle rules:

Security Artifact	Mobile Storage Mechanism	Lifecycle & Purpose
Access Token (JWT)	Volatile Memory	Short-lived token attached to HTTP headers. Validates Row Level Security (RLS) policies on every database request.
Refresh Token	Flutter Secure Storage (Keystore/Keychain)	Long-lived token safely encrypted on the device hardware. Used to silently regenerate expired Access Tokens.
Session Context	State Management (e.g., GetX/BLoC)	Caches the auth.uid() and profile metadata locally to prevent redundant database queries during UI rendering.

4.5. Cloud Orchestration & Automation

The backend architecture is fully decoupled from the frontend, running on an autonomous cloud environment responsible for data ingestion, AI processing, and system monitoring. This design enables continuous 24/7 operation without manual intervention.

4.5.1. AWS EC2 Deployment & Environment Setup

The system is deployed on an AWS EC2 instance to ensure continuous and reliable execution of background services.

- **Environment Isolation:** Dependencies and SDK libraries are managed inside isolated Python virtual environments to prevent version conflicts.
- **Process Management:** Background services are orchestrated to ensure automatic restart and high system reliability.
- **Secure Communication:** The instance communicates securely with the database using encrypted environment variables and service-role keys, bypassing client-side RLS limits for administrative operations.

4.5.2. Guardian Engine (Risk Management Logic)

The Guardian Engine is a critical background service responsible for enforcing user-defined risk protection rules. It continuously monitors user portfolios and applies automated actions to prevent excessive losses.

The engine operates on a strict logical loop:

1. **Querying Active Rules:** The script dynamically fetches all active protection rules and cross-references them with the user's current holdings.
2. **Threshold Evaluation:** For each holding, it compares the latest fetched market price against the user's average purchase price.
3. **Stop-Loss Evaluation:** The engine continuously checks if the asset's price has breached the liquidation threshold using the following mathematical condition:

$$\text{Current Price} < \text{Average Price} \left(1 - \left(\text{Liquidation Percentage} / 100 \right) \right)$$

4. **Automated Liquidation:** If the condition is met, the Guardian Engine instantly executes a liquidation request via a database RPC, ensuring real-time risk mitigation without requiring manual user intervention.

4.5.3. Automated Daemons & Market Status Tracker

The system deploys continuous, infinite-loop Python processes (Daemons) that independently monitor, fetch, and analyze market data without any manual triggering.

Daemon Name	Execution Frequency	Operational Objective
OHLC Ingestion Engine	Every 20 Seconds	Fetches real-time price ticks for Egyptian stocks, Gold, and Forex, subsequently updating the dynamically generated tables.
Market Status Tracker	Every 5 Minutes	Verifies EGX market hours. Pauses all other scraping operations during weekends or after closing hours to save compute/API costs.
News & AI Sentiment	Every 15 Minutes	Scrapes top financial portals, feeds the text to the Cerebras Llama 3.1 API and FinBERT for cleansing and sentiment labeling, and inserts it into the database.
Prediction Pipeline	Daily (Post-Market)	Runs predictive models on historical data to generate the next day's forecast, updating the predictions table.

CHAPTE

Title of Chapter

5.Title of Chapter 5

5.1. survey

CHAPTE

Title of Chapter

6.Title of Chapter 6

6.1. survey

CHAPTE

Conclusions

7. Conclusions

7.1. Survey

References



References